German International University

Faculty of Informatics and Computer Science

CNET101: Computer Networks — Spring 2026

GIU-Net: UDP Chat Application

& Campus Network Simulation — Project Report

Member Name	Student ID
Mazen Medhat	19003917
Kareem Ashour	19006115
Joumana Zahwy	19009029
Omar Wael	19009487

Instructor: Dr. Maggie Shammaa

Submission Date: May 31, 2026

1. Team Information

1.1 Team Members & Roles

Member Name	Student ID	Role	Last Digit
Member 1 (Team Leader)	19003917	Leader — lowest ID	7
Member 2	19006115	Member	5
Member 3	19009029	Member	9
Member 4	19009487	Member	7

1.2 Port Number Derivation

The server port is derived from the last 4 digits of the Team Leader's student ID.

1. Team Leader ID: 19003917
2. Extract last 4 digits: 3917
3. 3917 > 1024, so no adjustment needed.
4. **Server Port = 3917**

1.3 Handshake String Derivation

The verification sum is the arithmetic sum of the last digit of each team member's student ID.

5. Member 1 last digit: 7 (ID ends in ...3917)
6. Member 2 last digit: 5 (ID ends in ...6115)
7. Member 3 last digit: 9 (ID ends in ...9029)
8. Member 4 last digit: 7 (ID ends in ...9487)
9. **Sum = 7 + 5 + 9 + 7 = 28**

Resulting handshake strings:

```
Client sends:   HELLO-GIU-19003917-28
Server replies: OK-GIU-19003917-28
```

2. Part 1 — GIU-Net UDP Chat Implementation

2.1 Protocol Overview

GIU-Net is a connection-oriented chat application built entirely on raw UDP sockets (DatagramSocket / DatagramPacket) in Java. Since UDP provides no delivery guarantees, the protocol implements its own handshake, sequence numbers, acknowledgments, and retransmission logic at the application layer. No TCP classes (Socket, ServerSocket) are used anywhere in the implementation.

2.2 GIU-Net Packet Format

Byte(s)	Field	Size	Description
0–1	**Sequence Number**	2 bytes	16-bit unsigned integer, big-endian byte order. Set to 0 for handshake and control packets. Increments by 1 for each DATA packet.
2+	**Payload**	Variable	UTF-8 encoded text. Content and meaning depends on packet type (HELLO, OK, ACK, EXIT, or chat message).

2.3 Packet Types

Payload Content	Direction	Meaning
HELLO-GIU-19003917-28	Client → Server	Handshake initiation
OK-GIU-19003917-28	Server → Client	Handshake accepted
REJECTED	Server → Client	Handshake refused — wrong team string
ACK:<seqnum>	Either	Acknowledgment of a DATA packet with that sequence number
EXIT	Either	Request to close the connection
Any other text	Either	A chat message (DATA packet)

2.4 Protocol Design Choices

Retransmission Timeout

The retransmission timeout was set to 3 to account for real-world network variability when running across a mobile hotspot between two physical machines. A shorter timeout triggered false timeouts due to hotspot latency spikes, causing unnecessary retransmissions. The 7-second window provides enough time for the ACK to arrive under moderate congestion while still being short enough to detect a genuinely lost connection.

Handling ACK Wait vs Keyboard Input

The implementation uses two separate threads: a receiver thread that continuously listens for incoming packets (ACKs, messages, EXIT signals), and the main thread that reads from keyboard input (stdin). When the main thread calls sendWithRetry(), it temporarily sets a socket timeout to wait for the specific ACK. The receiver thread runs as a daemon thread. This separation ensures that incoming messages are printed to the screen even while the sending side is waiting for an ACK, and the user can type immediately after an ACK is received without blocking.

2.5 Debugging Story

During development, the server would occasionally print [TIMEOUT] Retrying... even when the client was running and had successfully received the message. ACK packets appeared to be missing from the server's perspective.

Initially, we suspected a race condition where the receiver thread consumed the ACK before sendWithRetry() could read it. We added detailed print statements to log every packet received on both threads.

After examining the logs, we confirmed that the receiver thread was indeed reading the ACK packet meant for sendWithRetry(). Because both threads share the same DatagramSocket, the receiver thread would sometimes call socket.receive() first and consume the ACK. The fix was to check inside the receiver thread whether the incoming message starts with "ACK:" — if so, the thread ignores it and lets the sendWithRetry() loop handle it through its own receive call. This resolved the issue completely.

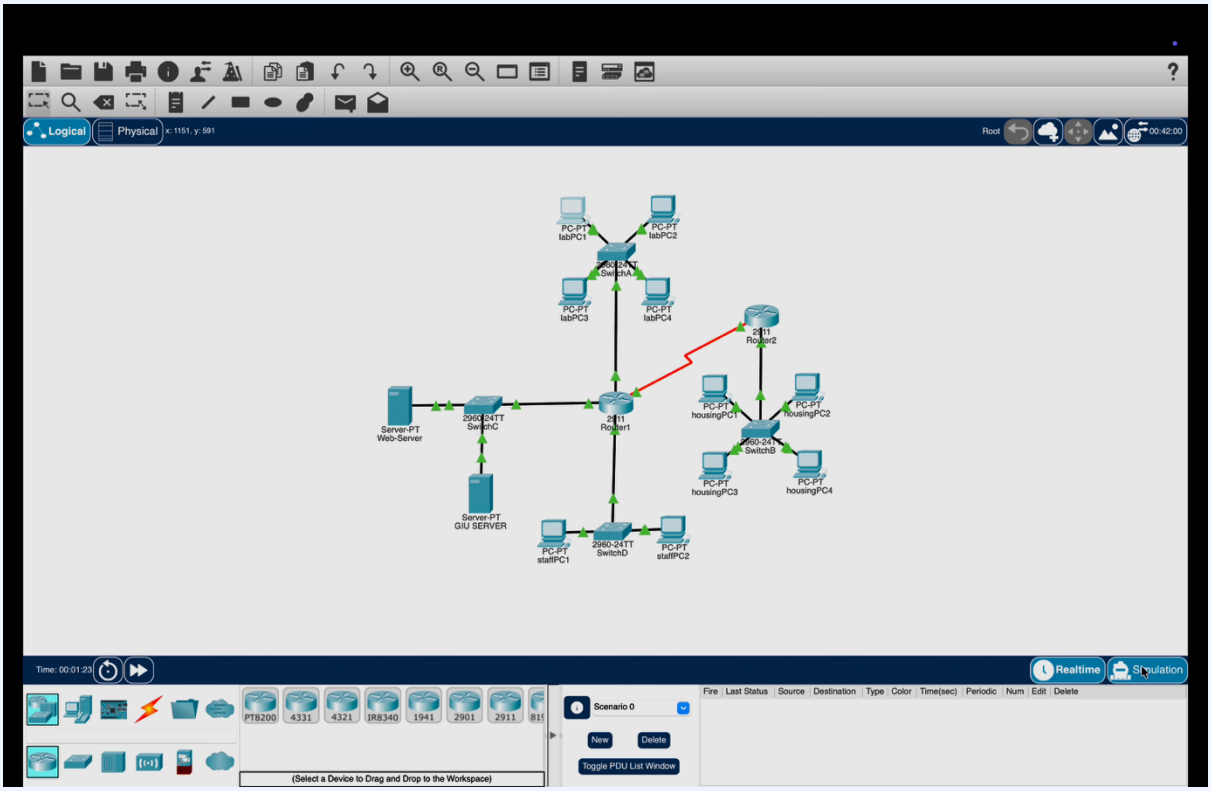
3. Part 2 — Campus Network Simulation (Cisco Packet Tracer)

3.1 Network Topology Overview

The GIU campus network was designed and simulated in Cisco Packet Tracer 9.0. It consists of two 2911 routers, four 2960 switches, ten PCs, and two servers interconnected to simulate realistic campus infrastructure.

Device	Model	Connected To	Subnet
Router 1	2911	Switch A, C, D + Serial WAN	B (Lab), A (Staff), E (DMZ), D (WAN)
Router 2	2911	Switch B + Serial WAN	C (Housing), D (WAN)
Switch A	2960	Router 1 + 4 Lab PCs	B — Student Lab
Switch B	2960	Router 2 + 4 Housing PCs	C — Student Housing
Switch C	2960	Router 1 + Web Server + GIU Server	E — Server DMZ
Switch D	2960	Router 1 + 2 Staff PCs	A — Staff Offices

Screenshot — Full Network Topology:



3.2 Cable Types

Connection	Cable Type	Notes
Router 1 Serial0/0/0 ↔ Router 2 Serial0/0/0	**Serial DCE/DTE**	Router 1 = DCE (clock rate 64000). WAN link, Subnet D.
Router 1/2 GigEthernet ↔ Switches	**Straight-through copper**	Router GigabitEthernet to Switch FastEthernet port
Switches ↔ PCs and Servers	**Straight-through copper**	PC/Server NIC to Switch FastEthernet port

3.3 Task 1 — VLSM Subnet Table

Base Network Derivation

The base network is derived from the Team Leader's student ID: 19003917.

10. 8-digit Leader ID: 19003917
11. Form IP from digit pairs: 19 . 00 . 39 . 0 → 19.0.39.0
12. Apply /22 prefix. /22 starts on a 4-block boundary: 19.0.36.0/22
13. Usable range: 19.0.36.0 – 19.0.39.255 (1022 usable hosts)

Subnets are allocated in order from largest to smallest required hosts to minimise address space waste:

Label	Purpose	Network Addr	Prefix	Subnet Mask	First IP	Last IP	Broadcast
C	Student Housing (200 hosts)	19.0.36.0	/24	255.255.255.0	19.0.36.1	19.0.36.254	19.0.36.255
B	Student Lab (100 hosts)	19.0.37.0	/25	255.255.255.128	19.0.37.1	19.0.37.126	19.0.37.127
A	Staff Offices (50 hosts)	19.0.37.128	/26	255.255.255.192	19.0.37.129	19.0.37.190	19.0.37.191
E	Server DMZ (10 hosts)	19.0.37.192	/28	255.255.255.240	19.0.37.193	19.0.37.206	19.0.37.207
D	WAN Link (2 hosts)	19.0.37.208	/30	255.255.255.252	19.0.37.209	19.0.37.210	19.0.37.211

Device IP Assignment

Device	IP Address	Subnet Mask	Default Gateway
Router 1 — Gi0/0 (Lab, Subnet B)	19.0.37.1	255.255.255.128	—
Router 1 — Gi0/1 (Staff, Subnet A)	19.0.37.129	255.255.255.192	—
Router 1 — Gi0/2 (DMZ, Subnet E)	19.0.37.193	255.255.255.240	—
Router 1 — Se0/0/0 (WAN)	19.0.37.209	255.255.255.252	—
Router 2 — Gi0/0 (Housing, Subnet C)	19.0.36.1	255.255.255.0	—
Router 2 — Se0/0/0 (WAN)	19.0.37.210	255.255.255.252	—
labPC1	19.0.37.2	255.255.255.128	19.0.37.1
labPC2	19.0.37.3	255.255.255.128	19.0.37.1
labPC3	19.0.37.4	255.255.255.128	19.0.37.1
labPC4	19.0.37.5	255.255.255.128	19.0.37.1
staffPC1	19.0.37.130	255.255.255.192	19.0.37.129
staffPC2	19.0.37.131	255.255.255.192	19.0.37.129
housingPC1	19.0.36.2	255.255.255.0	19.0.36.1
housingPC2	19.0.36.3	255.255.255.0	19.0.36.1
housingPC3	19.0.36.4	255.255.255.0	19.0.36.1
housingPC4	19.0.36.5	255.255.255.0	19.0.36.1
Web Server	19.0.37.194	255.255.255.240	19.0.37.193
GIU SERVER	19.0.37.195	255.255.255.240	19.0.37.193

3.4 Task 2 — Static Routing Configuration

Router 1 Routing Table

Router 1 has four directly connected interfaces (Subnets B, A, E, D). It needs one static route to reach Subnet C via Router 2:

```
ip route 19.0.36.0 255.255.255.0 19.0.37.210    ! To Student Housing via
Router 2
```

Full Router 1 CLI Configuration:

```
interface GigabitEthernet0/0
 ip address 19.0.37.1 255.255.255.128  ! Subnet B — Student Lab
 no shutdown
interface GigabitEthernet0/1
 ip address 19.0.37.129 255.255.255.192  ! Subnet A — Staff Offices
 no shutdown
interface GigabitEthernet0/2
 ip address 19.0.37.193 255.255.255.240  ! Subnet E — Server DMZ
 no shutdown
interface Serial0/0/0
 ip address 19.0.37.209 255.255.255.252  ! Subnet D — WAN
 clock rate 64000
 no shutdown
ip route 19.0.36.0 255.255.255.0 19.0.37.210
```

Router 2 Routing Table

Router 2 has two directly connected interfaces (Subnets C, D). It needs three static routes to reach all subnets on Router 1's side:

Full Router 2 CLI Configuration:

```
interface GigabitEthernet0/0
 ip address 19.0.36.1 255.255.255.0  ! Subnet C — Student Housing
 no shutdown
interface Serial0/0/0
 ip address 19.0.37.210 255.255.255.252  ! Subnet D — WAN
 no shutdown
ip route 19.0.37.0 255.255.255.128 19.0.37.209   ! Subnet B — Student Lab
ip route 19.0.37.128 255.255.255.192 19.0.37.209  ! Subnet A — Staff Offices
ip route 19.0.37.192 255.255.255.240 19.0.37.209  ! Subnet E — Server DMZ
```

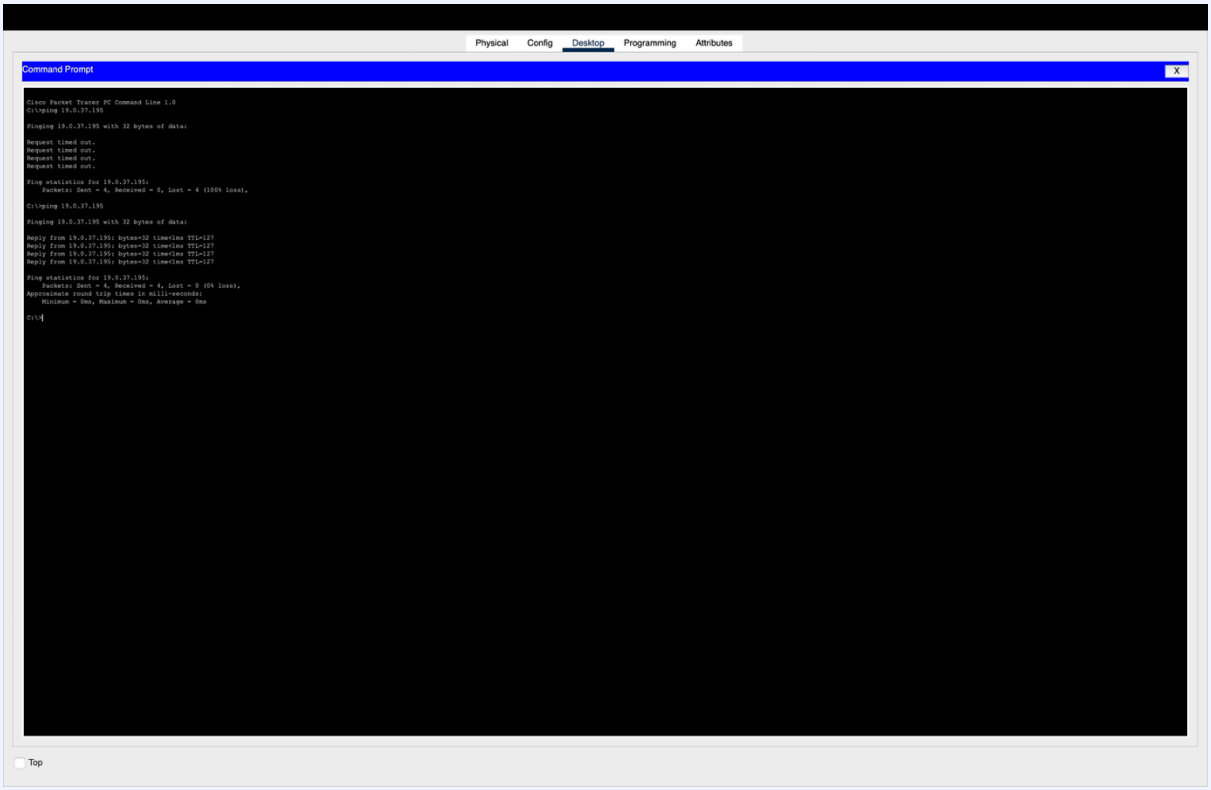
Connectivity Verification — housingPC1 to GIU SERVER

End-to-end connectivity was verified by pinging from housingPC1 (Subnet C) to GIU SERVER (Subnet E). This path crosses Router 2 → WAN serial link → Router 1, proving full inter-subnet routing:

```
C:\>ping 19.0.37.195
Reply from 19.0.37.195: bytes=32 time=2ms TTL=126
Reply from 19.0.37.195: bytes=32 time=3ms TTL=126
Reply from 19.0.37.195: bytes=32 time=11ms TTL=126
Reply from 19.0.37.195: bytes=32 time=9ms TTL=126
Packets: Sent = 4, Received = 4, Lost = 0 (100% success)
```

TTL value of 126 confirms 2 router hops traversed (128 - 2 = 126): Router 2 and Router 1.

Screenshot — Successful Ping housingPC1 to GIU SERVER:



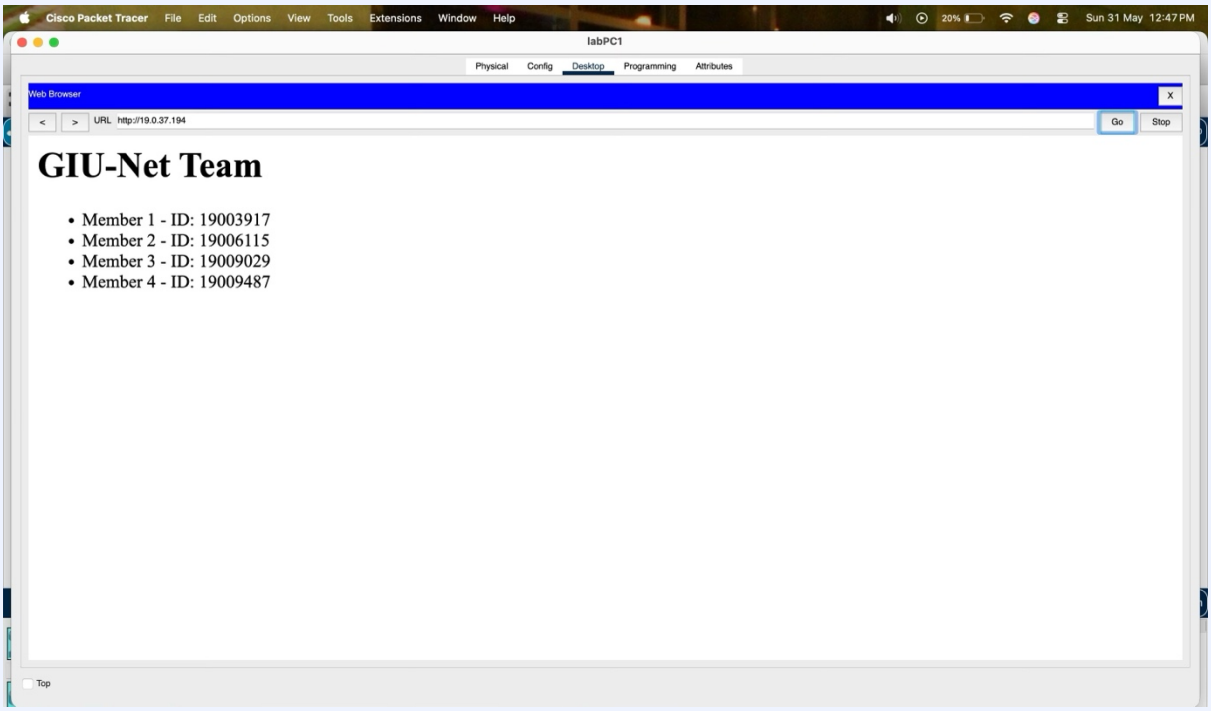
3.5 Task 3 — HTTP Service & Custom Team Webpage

The HTTP service was enabled on the Web Server (IP: 19.0.37.194) in Subnet E. The index.html file was customised to display the team name and all member student IDs. The page was successfully loaded from labPC1 by navigating to 19.0.37.194 in Packet Tracer's built-in browser.

Web Server index.html:

```
<html><body>
<h1>GIU-Net Team</h1>
<ul>
  <li>Member 1 - ID: 19003917</li>
  <li>Member 2 - ID: 19006115</li>
  <li>Member 3 - ID: 19009029</li>
  <li>Member 4 - ID: 19009487</li>
</ul>
</body></html>
```

Screenshot — Team Webpage Loaded in Browser:



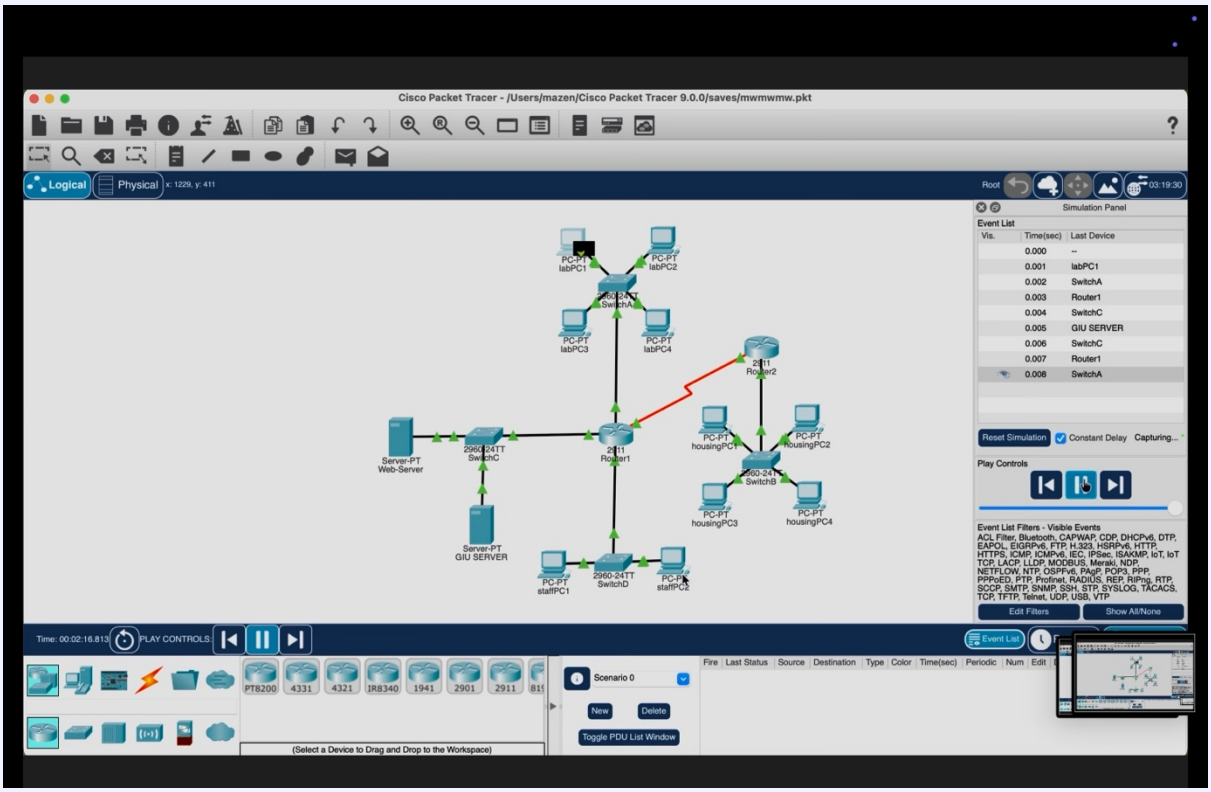
3.6 Task 4 — Simulation Mode Packet Trace

Using Cisco Packet Tracer's Simulation Mode with the ICMP filter enabled, a single ping request was traced from labPC1 (19.0.37.2, Subnet B) to GIU SERVER (19.0.37.195, Subnet E). The packet traversed 4 hops on the way there and 4 hops on the return path.

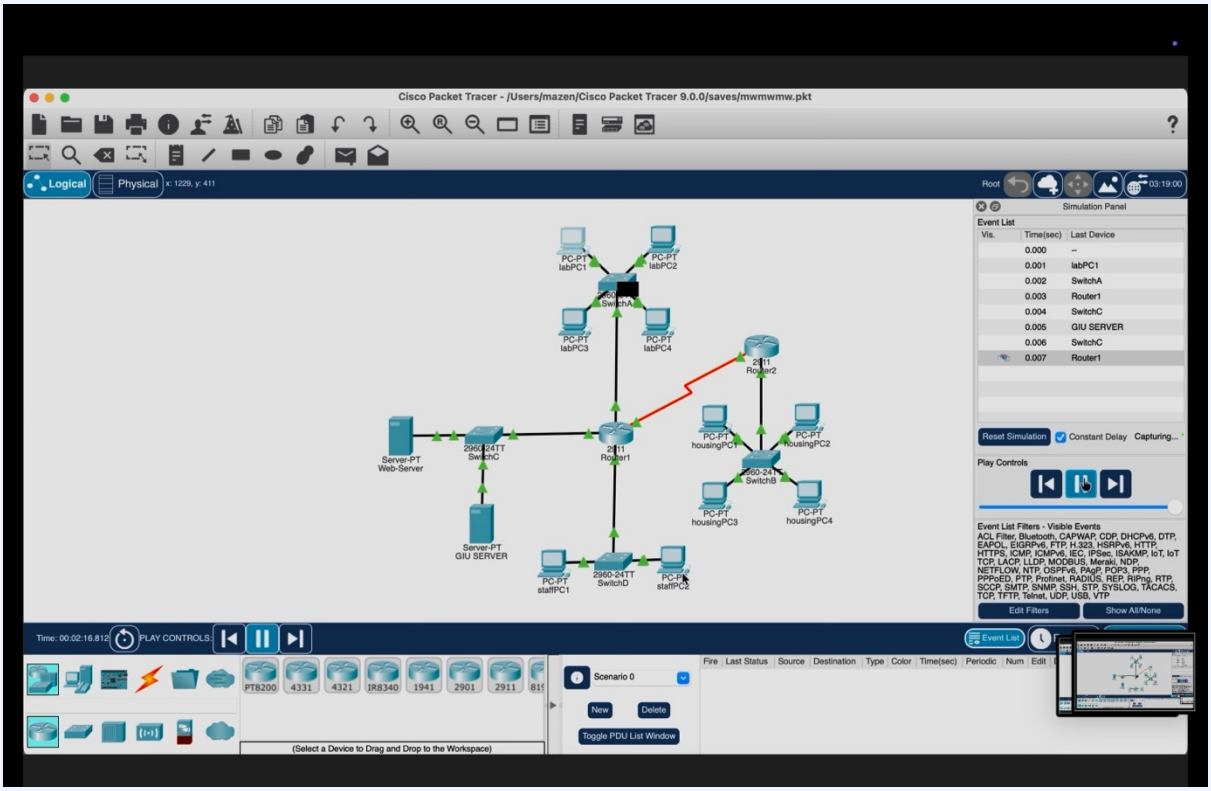
Hop	Device	Source IP	Dest IP	Layer Processing	Direction
1	**labPC1**	19.0.37.2	19.0.37.195	Network (IP) — ICMP packet created and encapsulated	Outgoing
2	**Switch A**	19.0.37.2	19.0.37.195	Data Link — MAC frame forwarded, no IP processing	Forward
3	**Router 1**	19.0.37.2	19.0.37.195	Network — IP packet decapsulated, route lookup performed, re-encapsulated with new MAC	Processing
4	**Switch C**	19.0.37.2	19.0.37.195	Data Link — MAC frame forwarded to GIU SERVER	Forward
5	**GIU SERVER**	19.0.37.2	19.0.37.195	Network — ICMP request received, reply packet generated	Destination

Screenshots — Simulation Mode Packet Trace (annotated):

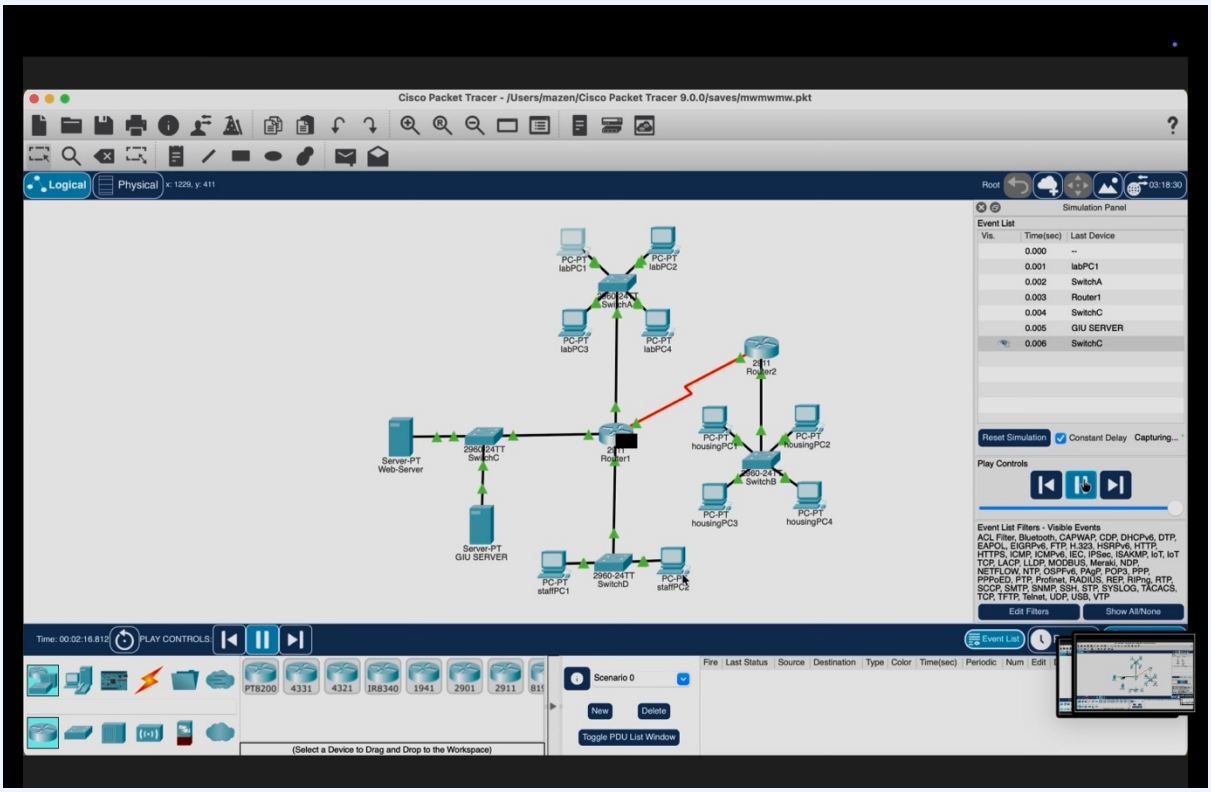
Hop 1 — labPC1 (Packet creation):



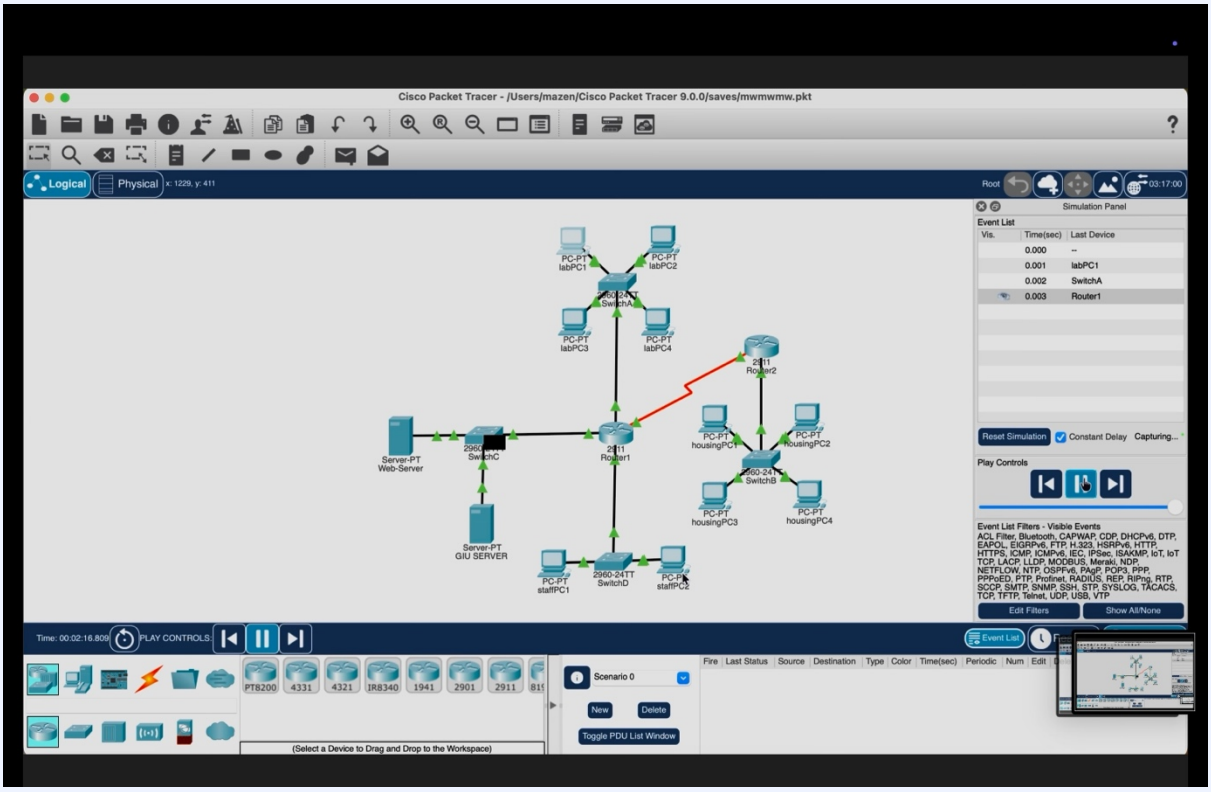
Hop 2 — Switch A (Data Link forwarding):



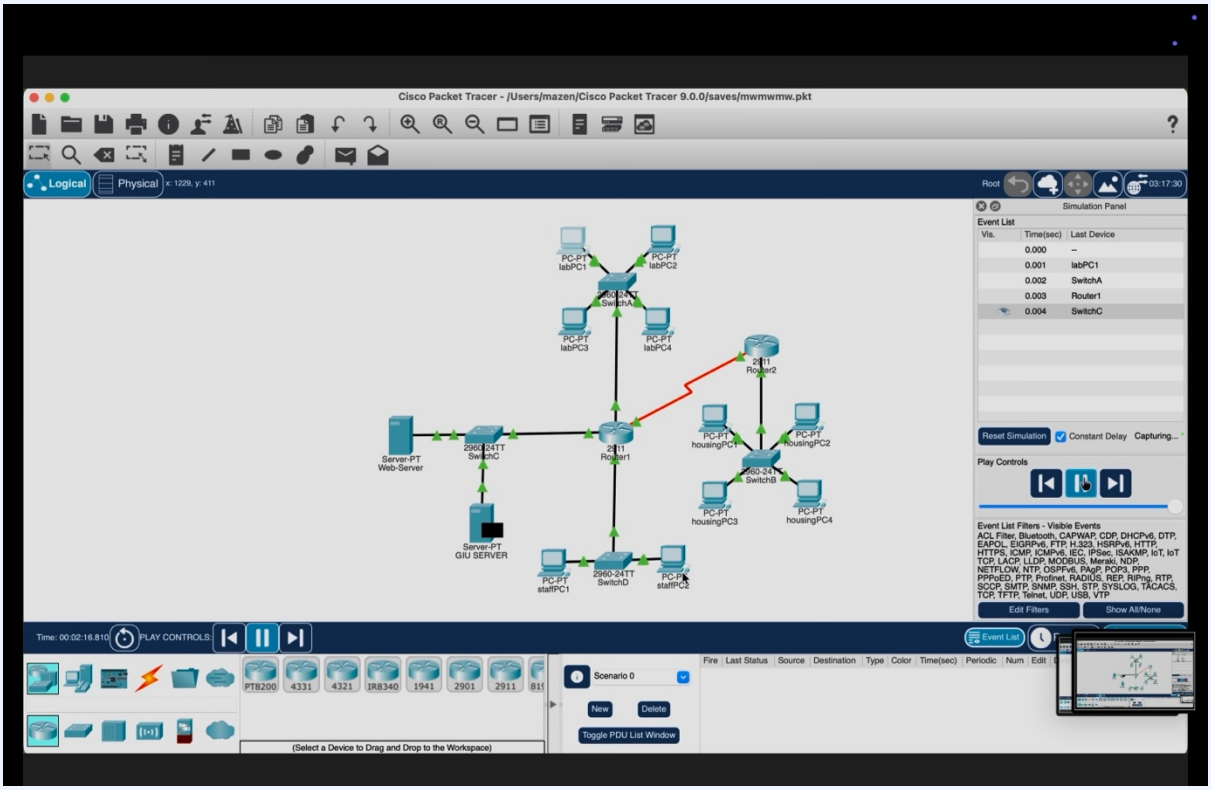
Hop 3 — Router 1:



Hop 4 — Switch C :



Hop 5 — GIU SERVER:



5. Wireshark Capture

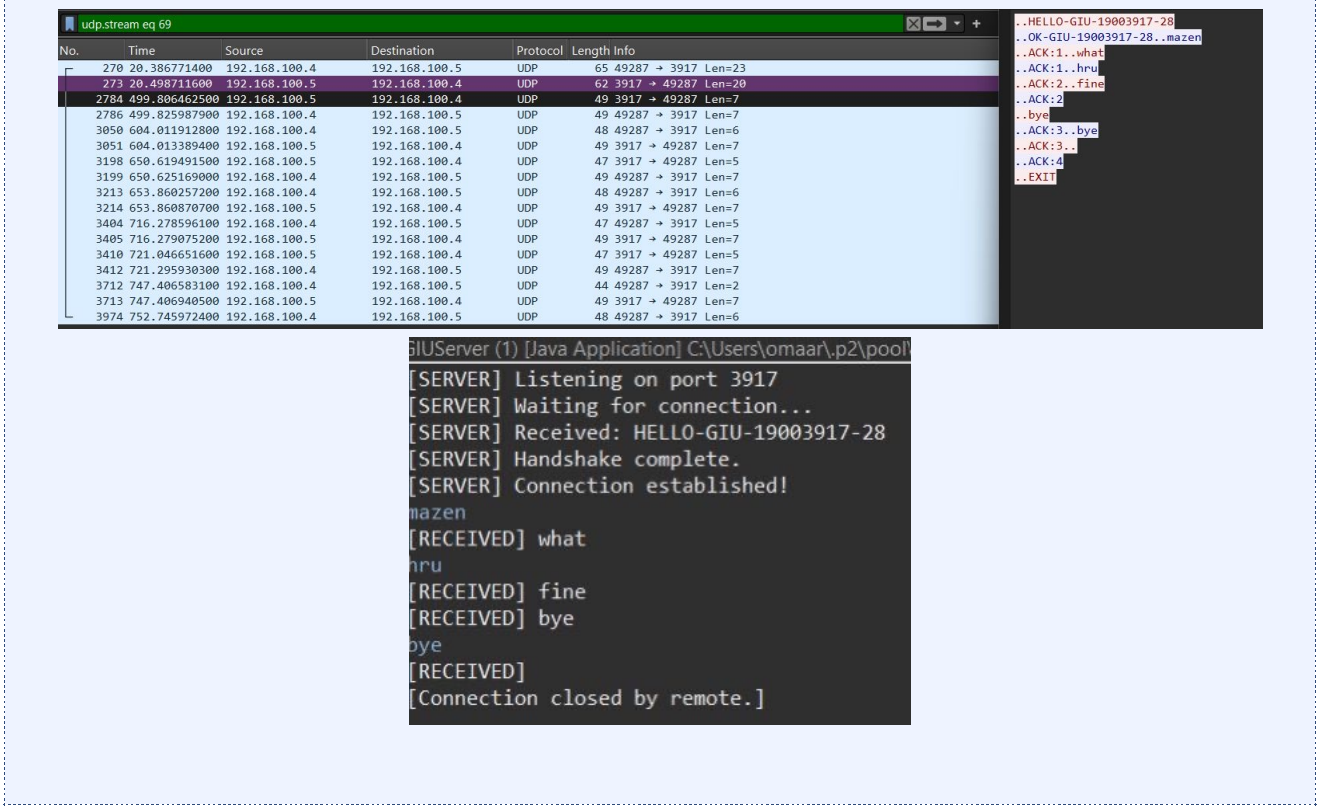
5.1 Capture Setup

Wireshark was run on the server machine during a live UDP chat session between two physical machines connected via a mobile hotspot. The capture was filtered to display only traffic on the server's UDP port:

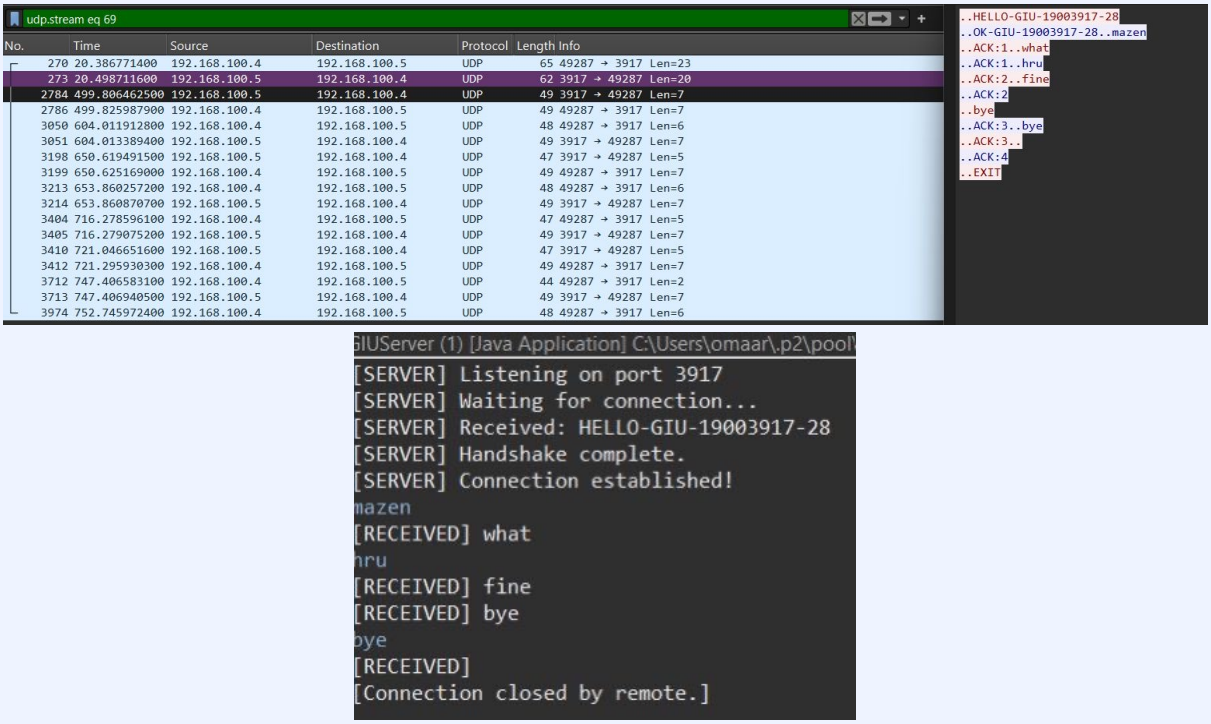
```
udp.port == 3917
```

5.2 Annotated Capture Screenshots

Screenshot — HELLO Handshake Packet:



Screenshot — OK Reply Packet:



Screenshot — DATA and ACK Packets:

